

Gestión de Proyecto

Proyecto: SISTEMA MULTIAGENTE PARA LA PRIORIZACIÓN DE PROYECTOS DE INVERSIÓN MUNICIPAL

Integrantes:

- Carhuamaca Vasquez, Diego
- Chuchon Sotelo, Ernesto
- Huachos Lopez, Madelyn
- Reinaga Cisneros, Hermiluz
- Sierra Leiva, Piero Martin

1. Necesidades de herramientas del proyecto

El proyecto requiere el soporte de herramientas especializadas en cada una de sus etapas: diseño de arquitectura, implementación del backend, orquestación de agentes de inteligencia artificial, persistencia de datos, procesamiento asíncrono, control de versiones, gestión del proyecto y verificación de calidad. La siguiente tabla identifica las actividades que demandan una herramienta específica y justifica la necesidad en cada caso.

Actividad del proyecto	Necesidad técnica	Categoría
Diseño de arquitectura hexagonal	Modelar componentes, puertos y adaptadores mediante diagramas UML que sirvan como contrato entre los módulos del sistema	Ingeniería y Modelado
Implementación de la API REST (Backend)	Desarrollar endpoints, validar esquemas de entrada/salida y exponer documentación Swagger de forma automática sin configuración adicional	Desarrollo y DevOps
Persistencia relacional (PostgreSQL)	Gestionar migraciones de esquema, transacciones ACID y persistir los checkpoints inmutables del grafo LangGraph ante interrupciones del flujo	Desarrollo y DevOps
Orquestación de agentes (LangGraph)	Modelar el pipeline de evaluación como máquina de estados determinista con primitivas de interrupción (interrupt) para la validación humana en el bucle (HITL)	Datos, IA y Modelado
Motor RAG Agéntico (Qdrant)	Indexar fragmentos de normativa municipal y recuperarlos mediante búsqueda semántica híbrida (vectorial densa + BM25 léxica) con alta precisión	Datos, IA y Modelado

Actividad del proyecto	Necesidad técnica	Categoría
Procesamiento asíncrono (Celery/Redis)	Desacoplar las tareas de cómputo intensivo del hilo de la API REST para garantizar un tiempo de bloqueo de la interfaz de usuario de 0 segundos	Desarrollo y DevOps
Gestión del backlog y los sprints	Planificar historias de usuario, asignar responsables y realizar seguimiento del avance en cada uno de los tres PMV bajo el marco Scrum adaptado	Gestión y Colaboración
Control de versiones y CI/CD	Mantener el historial de cambios, facilitar la integración continua entre los módulos y garantizar entornos de ejecución reproducibles mediante contenedores	Desarrollo y DevOps
Pruebas de la API y de carga	Verificar los contratos REST, detectar regresiones funcionales y medir el tiempo de respuesta del pipeline multiagente bajo carga concurrente simulada	Calidad y Seguridad
Experimentación con embeddings	Comparar configuraciones de modelos de embeddings y evaluar la precisión del motor RAG en un entorno con GPU gratuita antes de fijar el modelo definitivo en producción	Datos, IA y Modelado

1.1 Criterios de selección

Para garantizar la objetividad en la selección de herramientas se definieron los siguientes criterios ponderados, alineados con las restricciones reales del proyecto: coste cero, infraestructura académica y compatibilidad con el ecosistema Python/LangGraph/CrewAI. Las calificaciones técnicas de las matrices deben interpretarse junto con la documentación oficial citada en las referencias [1]-[5], [10]-[13].

Criterio	Peso	Justificación
Funcionalidad y cobertura del requerimiento	25 %	La herramienta debe satisfacer directamente la necesidad técnica identificada en la sección anterior
Integración con el ecosistema Python / LangGraph / Qdrant	20 %	La compatibilidad nativa reduce el esfuerzo de adaptación y el riesgo técnico del proyecto
Costo cero / licencia de código abierto	15 %	Restricción presupuestaria del proyecto académico: se utilizará exclusivamente infraestructura gratuita
Escalabilidad y rendimiento	15 %	La herramienta debe soportar múltiples agentes concurrentes y consultas asíncronas sin degradación
Seguridad y auditabilidad	10 %	Requerido por el componente de validación humana (HITL) y la trazabilidad de decisiones institucionales

Criterio	Peso	Justificación
Curva de aprendizaje del equipo	10 %	El equipo cuenta con 16 semanas; se priorizan herramientas con documentación activa y ejemplos disponibles
Usabilidad y soporte comunitario	5 %	Una comunidad activa reduce el tiempo de resolución de incidencias durante los sprints del proyecto

2. Selección y justificación de herramientas

A continuación se presentan las matrices de selección para las herramientas de mayor impacto en el proyecto. En cada caso se evaluaron al menos dos alternativas aplicando los criterios ponderados definidos en la sección anterior.

2.1 Framework de API REST

Criterio (peso)	FastAPI	Django REST	Flask	Justificación
Funcionalidad (25%)	5	4	3	FastAPI genera documentación Swagger/OpenAPI de forma automática y valida esquemas vía Pydantic, lo que resulta imprescindible para el contrato con el módulo de Frontend e Integración
Integración Python (20%)	5	3	4	Soporte nativo para async/await, compatible de forma directa con LangGraph y Celery sin adaptadores adicionales
Costo cero (15%)	5	5	5	Las tres alternativas son de código abierto; empate en este criterio
Escalabilidad (15%)	5	4	3	FastAPI con Uvicorn gestiona un alto número de conexiones concurrentes en modo asíncrono
Seguridad (10%)	4	5	3	Django incluye más utilidades de seguridad integradas; FastAPI requiere configurar OAuth2/JWT de forma explícita en el PMV3
Curva de aprendizaje (10%)	4	3	5	FastAPI es más conciso que Django; su documentación oficial es extensa y se mantiene actualizada
Usabilidad (5%)	5	4	4	FastAPI tiene la comunidad de crecimiento más rápido entre los frameworks Python modernos
Puntaje ponderado	4.80	3.80	3.85	FastAPI obtiene el mayor valor ponderado en el contexto del proyecto.

De acuerdo con los resultados obtenidos mediante la evaluación ponderada, se seleccionó FastAPI, debido a que obtuvo un valor ponderado de 4.80, frente a Flask (3.85) y Django REST Framework (3.80). Su elección responde principalmente a la validación tipada con Pydantic, la generación automática de OpenAPI y el soporte asíncrono requerido para integrar el backend con LangGraph, Qdrant y las tareas desacopladas del sistema multiagente.

2.2 Base de datos relacional

Criterio (peso)	PostgreSQL	MySQL	MongoDB	Justificación
Funcionalidad (25%)	5	4	3	PostgreSQL es el único motor con soporte nativo de checkpointing en LangGraph (PostgresSaver) y con tipo JSONB indexable para el estado del grafo
Integración Python (20%)	5	4	3	Driver oficial psycopg3, soporte de primer nivel en SQLAlchemy y Alembic para la gestión automática de migraciones
Costo cero (15%)	5	5	5	Las tres alternativas son de código abierto; empate en este criterio
Escalabilidad (15%)	4	4	5	PostgreSQL escala correctamente para el volumen del prototipo académico sin requerir infraestructura adicional
Seguridad/Auditabilidad (10%)	5	4	3	El modelo relacional con integridad referencial fuerte es el más adecuado para los registros de auditoría del HITL
Curva de aprendizaje (10%)	4	4	3	El equipo cuenta con experiencia previa en SQL; PostgreSQL es el motor estándar en

Criterio (peso)	PostgreSQL	MySQL	MongoDB	Justificación
				cursos anteriores de la carrera
Usabilidad (5%)	5	4	4	pgAdmin como herramienta visual; documentación oficial completa y activa
Puntaje ponderado	4.75	4.10	3.40	PostgreSQL obtiene el mayor valor ponderado en el contexto del proyecto.

De acuerdo con los resultados obtenidos mediante la evaluación ponderada, se seleccionó PostgreSQL, debido a que obtuvo un valor ponderado de 4.75, frente a MySQL (4.10) y MongoDB (3.40). Su elección responde principalmente a la integridad transaccional, el soporte JSONB, la integración con SQLAlchemy/Alembic y la disponibilidad de persistencia de checkpoints para LangGraph, aspectos necesarios para la trazabilidad del HITL y la arquitectura hexagonal del proyecto.

2.3 Base de datos vectorial

Criterio (peso)	Qdrant	Chroma / FAISS	Justificación
Funcionalidad (25%)	5	3	Qdrant soporta búsqueda híbrida densa+BM25 nativa, indexado jerárquico de chunks y filtros de payload, requeridos por el motor RAG Agéntico
Integración Python (20%)	5	4	Cliente oficial en Python con integración directa con LangChain/LangGraph y LlamaIndex
Costo cero (15%)	5	4	Qdrant Cloud dispone de un tier gratuito permanente; la versión dockerizada es local sin costo alguno
Escalabilidad (15%)	5	3	Qdrant está diseñada para entornos de producción con replicación; Chroma y FAISS son soluciones embebidas no aptas para concurrencia alta
Seguridad (10%)	4	2	Qdrant permite API keys y configuración TLS; FAISS no dispone de capa de seguridad propia
Curva de aprendizaje (10%)	4	4	Documentación oficial actualizada; SDK de Python con ejemplos de hybrid search disponibles

Criterio (peso)	Qdrant	Chroma / FAISS	Justificación
Usabilidad (5%)	5	3	Dashboard web integrado para inspección de colecciones y vectores sin necesidad de herramientas externas
Puntaje ponderado	4.80	3.35	Qdrant obtiene el mayor valor ponderado para el motor RAG Agéntico.

De acuerdo con los resultados obtenidos mediante la evaluación ponderada, se seleccionó Qdrant, debido a que obtuvo un valor ponderado de 4.80, frente a Chroma/FAISS (3.35). Su elección responde principalmente a la búsqueda híbrida con representaciones densas y dispersas, filtros por metadatos, API de consulta multietapa e integración con Python, capacidades alineadas con el RAG Agéntico que utilizarán los agentes del sistema para recuperar normativa municipal de forma trazable [4].

2.4 Selección de herramientas de Inteligencia Artificial

Las tecnologías de inteligencia artificial consideradas cumplen funciones distintas. LangGraph y CrewAI pertenecen a la capa de orquestación y coordinación de agentes, mientras que OpenAI API y Gemini API son proveedores de modelos generativos. Por ello, la evaluación se realiza en dos matrices independientes y no se comparan como herramientas equivalentes.

Herramienta	Tipo	Función en el proyecto	Ventajas	Limitaciones
LangGraph	Orquestador de agentes	Controlar el flujo del grafo, checkpoints, persistencia e interrupciones HITL.	Estado explícito, persistencia y control del flujo.	Mayor complejidad conceptual y API en evolución.
CrewAI	Framework multiagente	Definir agentes, roles, tareas y coordinación especializada.	Modelo declarativo de agentes y rápida composición de equipos.	Menor control fino del estado y del flujo HITL que LangGraph.
OpenAI API	Proveedor/modelos LLM	Generar dictámenes estructurados y ejecutar herramientas desde los agentes.	SDK Python, tool calling y salidas estructuradas.	Servicio de pago sujeto a cuotas y límites de tasa.
Gemini API	Proveedor/modelos LLM	Alternativa/fallback para generación y	SDK Python, function calling y capa gratuita en determinados	Cuotas variables y diferencias de

Herramienta	Tipo	Función en el proyecto	Ventajas	Limitaciones
		llamadas a funciones.	modelos/condiciones.	comportamiento entre modelos.

2.4.1 Evaluación ponderada de orquestadores multiagente

Criterio	Peso	LangGraph	Valor ponderado	CrewAI	Valor ponderado
Funcionalidad y cobertura del requerimiento	25 %	5	1.25	4	1.00
Integración con Python/LangGraph/Qdrant	20 %	5	1.00	4	0.80
Costo cero/licencia de código abierto	15 %	5	0.75	5	0.75
Escalabilidad y rendimiento	15 %	4	0.60	4	0.60
Seguridad y auditabilidad	10 %	5	0.50	3	0.30
Curva de aprendizaje del equipo	10 %	3	0.30	4	0.40
Usabilidad y soporte comunitario	5 %	4	0.20	4	0.20
TOTAL	100 %		4.60		4.05

La matriz asigna a LangGraph un valor ponderado de 4.60 y a CrewAI 4.05. LangGraph obtiene ventaja por su control explícito del estado, persistencia y mecanismos de interrupción adecuados para flujos HITL; estas capacidades son relevantes para conservar checkpoints y auditar el proceso de priorización. CrewAI mantiene valor como capa complementaria para la definición de roles, tareas y colaboración entre agentes, pero no se utilizará como orquestador principal del flujo. En consecuencia, LangGraph queda seleccionado mediante matriz ponderada como orquestador principal y CrewAI se adopta como soporte complementario donde aporte expresividad al diseño de los agentes [3][11].

2.4.2 Evaluación ponderada de proveedores/modelos LLM

Criterio	Peso	OpenAI API	Valor ponderado	Gemini API	Valor ponderado
Funcionalidad y cobertura del requerimiento	25 %	5	1.25	4	1.00
Integración con Python/LangGraph/Qdrant	20 %	5	1.00	4	0.80
Costo cero/licencia de código abierto	15 %	2	0.30	5	0.75

Criterio	Peso	OpenAI API	Valor ponderado	Gemini API	Valor ponderado
Escalabilidad y rendimiento	15 %	5	0.60	4	0.75
Seguridad y auditabilidad	10 %	4	0.40	4	0.40
Curva de aprendizaje del equipo	10 %	4	0.40	4	0.40
Usabilidad y soporte comunitario	5 %	5	0.25	5	0.25
TOTAL	100 %		4.20		4.35

La evaluación otorga 4.35 a OpenAI API y 4.25 a Gemini API. OpenAI API obtiene una ventaja pequeña por la madurez de su SDK, el soporte de llamadas a herramientas y salidas estructuradas que facilitan producir dictámenes JSON consistentes para el backend [10][12]. Su principal limitación para este proyecto académico es el costo, por lo que no se considera una dependencia exclusiva. Gemini API obtiene la máxima calificación en costo y ofrece function calling desde Python [13]; se mantiene como alternativa/fallback y como opción de experimentación cuando sus cuotas gratuitas resulten suficientes. Por tanto, OpenAI API queda como servicio principal según la matriz, mientras Gemini API se adopta como alternativa operativa y de contingencia.

2.5 Resumen de herramientas seleccionadas

Categoría	Alternativas evaluadas	Herramienta seleccionada	Puntaje ponderado	Justificación principal
Framework API REST	FastAPI, Django REST Framework, Flask	FastAPI	4.80	API asíncrona, validación tipada y OpenAPI para integrar los módulos del sistema.
Base de datos relacional	PostgreSQL, MySQL, MongoDB	PostgreSQL	4.75	Persistencia transaccional, JSONB, SQLAlchemy/Alembic y checkpoints del flujo HITL.
Base de datos vectorial	Qdrant, Chroma/FAISS	Qdrant	4.80	Búsqueda híbrida y filtros de metadatos para el RAG Agéntico.
Orquestación multiagente	LangGraph, CrewAI	LangGraph (principal)	4.60	Control determinista del grafo, estado, persistencia

Categoría	Alternativas evaluadas	Herramienta seleccionada	Puntaje ponderado	Justificación principal
				interrupciones HITL.
Proveedor/modelo LLM	OpenAI API, Gemini API	OpenAI API (principal)	4.35	Tool calling y salidas estructuradas; Gemini queda como fallback por costo/cuotas.

Las demás tecnologías descritas en la sección 3 —Python, Uvicorn, SQLAlchemy, Alembic, Celery, Redis, Docker, Docker Compose, Git, GitHub, herramientas de modelado, gestión, pruebas y seguridad— se consideran herramientas adoptadas como soporte del proyecto. Su inclusión responde a necesidades técnicas y operativas específicas, pero no debe interpretarse como resultado de una competencia ponderada cuando el documento no presenta alternativas equivalentes evaluadas.

3. Herramientas seleccionadas por categoría

En esta sección se describen las herramientas seleccionadas organizadas en cinco categorías: ingeniería y modelado, desarrollo e integración, datos e inteligencia artificial, gestión y colaboración, y calidad y seguridad. Para cada herramienta se indica su rol en el proyecto, la versión utilizada, el uso específico y la limitación reconocida.

3.1 Herramientas de ingeniería y modelado

Herramienta	Versión / Licencia	Uso en el proyecto	Limitación reconocida
PlantUML / Draw.io	PlantUML 1.2024.x / MIT; Draw.io web gratuito	Diagramas UML de componentes (arquitectura hexagonal), diagramas de despliegue de los contenedores Docker y diagramas de secuencia del flujo LangGraph	Draw.io no versiona automáticamente los diagramas; los archivos .puml y .drawio se gestionan manualmente en el repositorio GitHub
C4 Model (Structurizr Lite)	Structurizr Lite gratuito	Vista de Contexto (C4-L1), Vista de Contenedores (C4-L2) y Vista de Componentes internos del Backend (C4-L3) para comunicar la arquitectura hexagonal al equipo	Structurizr Lite no genera diagramas en tiempo real ante cambios de código; requiere actualización manual del modelo DSL ante cada refactorización

Herramienta	Versión / Licencia	Uso en el proyecto	Limitación reconocida
BPMN (Camunda Modeler)	Camunda Modeler 5.x / Apache 2.0	Modelado del proceso de evaluación municipal: registro de proyecto, evaluación multidimensional por agentes, punto de interrupción HITL y generación del ranking final	Camunda Modeler no simula los flujos directamente; la validación del proceso se realiza a través de las pruebas de integración del Backend

3.2 Herramientas de desarrollo, integración y DevOps

Herramienta	Versión / Licencia	Uso en el proyecto	Limitación reconocida
Python 3.12	3.12 / PSF License	Lenguaje base de todo el Backend y del módulo de agentes. Compatibilidad directa con LangGraph, CrewAI, psycpg3, SQLAlchemy y Celery sin adaptadores adicionales	Las dependencias de terceros pueden introducir incompatibilidades al actualizar versiones minor; se mitiga fijando la versión en requirements.txt
FastAPI + Uvicorn	FastAPI 0.115+ / MIT	Implementación de los endpoints REST: /proyectos, /evaluaciones y /ranking. Generación automática de documentación OpenAPI en /docs. Validación de esquemas mediante Pydantic v2	FastAPI no incluye autenticación integrada; se requiere configurar OAuth2/JWT de forma explícita en el PMV3
SQLAlchemy 2.x + Alembic	SQLAlchemy 2.0 / MIT	Definición de los modelos ORM (Proyecto, Evaluacion, Checkpoint). Gestión de migraciones de esquema sin tiempo de inactividad. Soporte para psycpg3 asíncrono	Las migraciones en entornos multi-contenedor pueden generar condiciones de carrera; se mitiga ejecutando alembic upgrade head en el entypoint del contenedor
Docker + Docker Compose	Docker 27.x / Apache 2.0	Contenedores independientes para FastAPI, PostgreSQL, Qdrant, Redis y el worker de Celery. El archivo docker-compose.yml garantiza un entorno de	Docker Desktop consume recursos significativos en equipos con menos de 8 GB de RAM; se usa el modo WSL2 en Windows para

Herramienta	Versión / Licencia	Uso en el proyecto	Limitación reconocida
		desarrollo reproducible para todos los integrantes del equipo	reducir la sobrecarga de memoria
Git + GitHub	Git 2.x / GPLv2; GitHub Free	Control de versiones con ramas por módulo (feature/backend-*, feature/ia-*). Pull requests con revisión cruzada entre Backend e Integración. GitHub Actions para análisis estático automático	El tier gratuito de GitHub Actions tiene un límite de 2 000 minutos por mes; se priorizan los workflows de lint y pruebas unitarias para no superar ese límite
Celery 5.x + Redis 7.x	Celery 5.4 / BSD; Redis 7 / BSD	Delegación al 100% de las tareas de cómputo intensivo fuera del hilo de la API REST, garantizando un tiempo de bloqueo de la interfaz de usuario de 0 segundos. Se implementa a partir del PMV2	Redis en modo standalone no ofrece persistencia garantizada ante fallos de nodo; se habilitará RDB + AOF persistence en el PMV3 para proteger el estado de las colas

3.3 Herramientas de datos, inteligencia artificial y modelado

Herramienta	Versión / Licencia	Uso en el proyecto	Limitación reconocida
LangGraph	0.2.x / MIT	Orquestación del pipeline de evaluación como grafo dirigido con nodos (Coordinador, Económico, Jurídico, Social, Ambiental, Técnico) y primitivas interrupt() para el checkpoint HITL. PostgresSaver para persistencia inmutable del estado	La API de LangGraph cambia frecuentemente entre versiones minor; se fija la versión en requirements.txt y se ejecutan pruebas de regresión antes de cada actualización
CrewAI	0.100+ / MIT	Declaración de los agentes especializados con sus roles, herramientas disponibles y objetivos.	CrewAI no ofrece primitivas de interrupción nativas; se combina con LangGraph para el control de flujo determinista y la persistencia de estado ante el HITL

Herramienta	Versión / Licencia	Uso en el proyecto	Limitación reconocida
		Complementa a LangGraph proporcionando la expresividad de roles y la lógica de coordinación interna entre agentes	
Qdrant	1.10+ / Apache 2.0	Almacenamiento e indexado jerárquico de los fragmentos normativos (chunks Padre e Hijo) con embeddings densos y representación BM25. Búsqueda semántica híbrida utilizada por el Agente Jurídico y el Agente Coordinador	El tier gratuito de Qdrant Cloud tiene un límite de 1 GB de almacenamiento vectorial, suficiente para el prototipo académico pero insuficiente para la normativa municipal completa
OpenAI API / Gemini API	Servicios API / documentación vigente	Generación de dictámenes estructurados por los agentes. OpenAI API se utiliza como servicio principal; Gemini API se mantiene como alternativa/fallback para experimentación y continuidad operativa.	Ambos servicios externos están sujetos a cuotas, límites de tasa y cambios de disponibilidad/modelos; el adaptador IIAService debe desacoplar al sistema del proveedor concreto.
Google Colab	Web / Gratuito (GPU T4)	Experimentación con modelos de embeddings (text-embedding-3-small, BAAI/bge-m3) para comparar la precisión del RAG antes de fijar el modelo definitivo. Evaluación de métricas de relevancia sobre el subconjunto normativo	Las sesiones de Colab se desconectan tras 12 horas de inactividad; los artefactos de experimentación se guardan en Google Drive y los notebooks se versionan en GitHub
LangSmith	Developer plan gratuito	Monitoreo de las trazas de ejecución del pipeline multiagente, registro	El plan gratuito almacena trazas durante 7 días; se exportan periódicamente a un archivo JSON para

Herramienta	Versión / Licencia	Uso en el proyecto	Limitación reconocida
		de prompts y respuestas, medición de latencia por nodo y evaluación de las métricas G-Eval y Answer Relevancy definidas en los criterios de aceptación	garantizar el registro de auditoría del HITL más allá de ese plazo
Pydantic v2	2.x / MIT	Definición de los modelos de datos tipados que estructuran los dictámenes JSON de cada agente (puntaje, viabilidad, justificación). Garantiza contratos de datos consistentes entre el Backend y el módulo de inteligencia artificial	La migración de Pydantic v1 a v2 introduce cambios incompatibles; se documenta la versión mínima requerida en requirements.txt para evitar regresiones

3.4 Herramientas de gestión y colaboración

Herramienta	Práctica de Scrum	Uso en el proyecto	Resultado demostrable
Trello (plan gratuito)	Product Backlog y Sprint Backlog	Tablero con columnas Backlog, Sprint, En Progreso, En Revisión y Hecho. Tarjetas por historia de usuario con responsable, PMV asignado y criterios de aceptación	Tablero público compartido con el docente-asesor; captura del estado al cierre de cada sprint y porcentaje de historias completadas frente a las planificadas
GitHub Projects + Issues	Seguimiento de issues y milestones por PMV	Issues vinculados a los pull requests de cada módulo. Milestones: PMV1 (S1-S4), PMV2 (S5-S10), PMV3 (S11-S16). Etiquetas: backend, ia, frontend, integración, bug, documentación	Historial de commits con mensajes semánticos (feat:, fix:, docs:, test:); gráfico de progreso generado por GitHub Insights al cierre de cada PMV
Discord (servidor del equipo)	Daily Scrum y comunicación asíncrona	Canales temáticos por módulo (#backend, #ia-agentes, #frontend, #integracion). Hilo por	Capturas de los acuerdos y decisiones registrados en los

Herramienta	Práctica de Scrum	Uso en el proyecto	Resultado demostrable
		sprint para registrar acuerdos y decisiones técnicas. Reuniones de revisión y retrospectiva por videoconferencia	hilos del servidor; actas de retrospectiva al cierre de cada PMV

3.5 Herramientas de calidad, pruebas y seguridad

Herramienta	Tipo de prueba	Uso en el proyecto	Métrica objetivo
Pytest + pytest-asyncio	Pruebas unitarias e integración del Backend	Pruebas unitarias de los casos de uso (registrar_proyecto, ejecutar_evaluacion) con mocks del repositorio. Pruebas de integración de los endpoints REST con el TestClient de FastAPI	Cobertura de código $\geq 80\%$ al cierre del PMV1; 0 pruebas fallidas en el pipeline de integración continua
Postman / Bruno	Pruebas de contratos REST	Colección de peticiones HTTP para todos los endpoints con aserciones automáticas sobre el código de estado, el esquema JSON y los tiempos de respuesta. Colección compartida entre los módulos de Backend e Integración	100 % de endpoints documentados con al menos un caso exitoso y un caso de error; tiempo de respuesta p95 menor a 2 segundos en el flujo de registro
SonarCloud (Free)	Análisis estático de calidad del código	Análisis automático en cada pull request: detección de bugs, vulnerabilidades, code smells y deuda técnica. Integrado con GitHub Actions. Quality Gate configurado para bloquear merges con incidencias críticas	0 code smells críticos al cierre de cada PMV; cobertura reportada igual o superior a la medida con Pytest
Locust / k6	Pruebas de carga y rendimiento del pipeline	Simulación de carga concurrente sobre los endpoints de evaluación en el PMV2 y el PMV3. Escenario de prueba: 10 usuarios concurrentes enviando 3 proyectos simultáneos al pipeline multiagente	Tiempo de bloqueo de la interfaz: 0 s (verificado en PMV2); latencia P95 del dictamen final menor a 60 s (tolerancia definida en el Project Charter)

Herramienta	Tipo de prueba	Uso en el proyecto	Métrica objetivo
OWASP ZAP (PMV3)	Análisis de vulnerabilidades de seguridad	Escaneo automatizado de los endpoints de la API en el PMV3 para detectar vulnerabilidades OWASP Top 10: inyección SQL, IDOR, exposición de datos sensibles y ausencia de autenticación en rutas protegidas	0 vulnerabilidades de nivel Alto o Crítico antes de la presentación final del PMV3

4. Aplicación integrada de herramientas en los tres PMV

Las herramientas no se utilizan de forma aislada sino como un ecosistema integrado. Cada PMV representa un incremento de madurez técnica que incorpora nuevas herramientas conforme aumenta la complejidad de la solución. El flujo de trabajo del proyecto sigue la cadena: problema documentado → modelado UML y BPMN → diseño de arquitectura en C4 → backlog en Trello → implementación en VS Code → commits en GitHub → contenedores Docker → pruebas con Pytest y Postman → análisis en SonarCloud → monitoreo en LangSmith → demo al docente-asesor → retroalimentación registrada en Discord → backlog ajustado.

Herramienta	PMV1 (S1-S4)	PMV2 (S5-S10)	PMV3 (S11-S16)	Actividad que sustenta
PlantUML / C4 Model / BPMN	Si	Si	Si	Diseño y comunicación de la arquitectura
Python 3.12	Si	Si	Si	Implementación de todo el Backend
FastAPI + Uvicorn	Si	Si	Si	API REST: registro, consulta y evaluación
SQLAlchemy 2.x + Alembic	Si	Si	Si	ORM y migraciones de PostgreSQL
Docker + Docker Compose	Si	Si	Si	Entorno de desarrollo reproducible
Git + GitHub	Si	Si	Si	Control de versiones y pull requests
LangGraph 0.2.x	Si	Si	Si	Orquestación como máquina de estados determinista
CrewAI 0.100+	Si	Si	Si	Declaración y coordinación de

Herramienta	PMV1 (S1-S4)	PMV2 (S5-S10)	PMV3 (S11-S16)	Actividad que sustenta
				la tripulación de agentes
Qdrant (vectorial)	Si	Si	Si	Motor RAG Agéntico sobre normativa municipal
OpenAI API / Gemini API	Servicios API / documentación vigente	Generación de dictámenes estructurados por los agentes. OpenAI API se utiliza como servicio principal; Gemini API se mantiene como alternativa/fallback para experimentación y continuidad operativa.	Ambos servicios externos están sujetos a cuotas, límites de tasa y cambios de disponibilidad/modelos; el adaptador IIAService debe desacoplar al sistema del proveedor concreto.	LLM de los agentes especializados
Google Colab	Si	Si	Parcial	Experimentación con embeddings y configuración del RAG
LangSmith (Developer)	Parcial	Si	Si	Observabilidad y métricas G-Eval / Answer Relevancy
Pydantic v2	Si	Si	Si	Validación de esquemas de los dictámenes de los agentes
Celery 5.x + Redis 7.x	No	Si	Si	Procesamiento asíncrono a partir del PMV2
Trello + GitHub Projects	Si	Si	Si	Product Backlog, sprints y seguimiento de milestones
Discord (servidor del equipo)	Si	Si	Si	Daily Scrum y registro de decisiones técnicas

Herramienta	PMV1 (S1-S4)	PMV2 (S5-S10)	PMV3 (S11-S16)	Actividad que sustenta
Pytest + pytest-asyncio	Si	Si	Si	Pruebas unitarias e integración continua
Postman / Bruno	Si	Si	Si	Pruebas de contratos REST
SonarCloud (análisis estático)	Parcial	Si	Si	Quality Gate en pull requests
Locust / k6 (carga)	No	Si	Si	Pruebas de rendimiento del pipeline multiagente
OWASP ZAP (seguridad)	No	No	Si	Análisis de vulnerabilidades OWASP Top 10

5. Resultados, limitaciones y decisiones de adaptación

Esta sección cumple de forma explícita con el verbo reconocer limitaciones de la competencia AG-I11 de ICACIT. Para cada herramienta crítica se documenta la limitación identificada durante la ejecución del proyecto, el impacto potencial y la acción de mitigación adoptada o planificada.

5.1 Resultados cuantificables por herramienta al cierre del PMV1

Herramienta	Indicador medido	Resultado al cierre del PMV1	Acción tomada
FastAPI (endpoints REST)	Porcentaje de endpoints documentados en Swagger	100 % (5 endpoints del flujo principal)	Mantener; ampliar con las rutas del ranking en el PMV3
PostgreSQL + Alembic	Migraciones aplicadas sin errores	3 migraciones exitosas; 0 rollbacks	Ampliar el esquema en el PMV2 con las tablas de checkpoint HITL
Docker Compose	Tiempo de levantamiento del entorno local	Aproximadamente 45 s en un equipo estándar	Optimizar con imágenes precompiladas en el PMV2
Pytest	Cobertura de código del Backend	83 % (meta definida: >= 80 %)	Ampliar la cobertura de los casos de error y excepciones de dominio en el PMV2
Postman	Pruebas de API aprobadas sobre el total	18 de 18 pruebas (100 %)	Automatizar la colección en GitHub

Herramienta	Indicador medido	Resultado al cierre del PMV1	Acción tomada
			Actions para el PMV2
LangGraph + PostgresSaver	Checkpoints persistidos correctamente	3 de 3 pruebas de persistencia HITL exitosas	Escalar a los 6 nodos de agente en el PMV2
Qdrant (RAG Agéntico)	Precisión de recuperación semántica (umbral > 94 %)	91.3 % con el subconjunto reducido de normativa en el PMV1	Ampliar el corpus e implementar re-ranking en el PMV2 para alcanzar el umbral
SonarCloud	Code smells críticos detectados	0 críticos; 7 smells menores resueltos en el sprint	Mantener el Quality Gate estricto en el PMV2 y el PMV3

5.2 Matriz de limitaciones y mitigaciones

Herramienta	Limitación identificada	Impacto potencial	Acción de mitigación adoptada
FastAPI	No incluye autenticación ni gestión de sesiones integradas	Endpoints desprotegidos en producción sin configuración adicional	Implementar OAuth2 con JWT (python-jose) y middleware de autorización por roles en el PMV3
PostgreSQL	En modo contenedor sin réplica, un fallo del nodo implica pérdida del estado activo	Pérdida de los checkpoints HITL que estén en vuelo al momento del fallo	Habilitar volúmenes Docker persistentes y snapshot diario de la base de datos durante el desarrollo
Qdrant Free tier	Límite de 1 GB de almacenamiento vectorial en Qdrant Cloud	Corpus de normativa incompleto en un escenario de producción real	Uso de la versión dockerizada local sin límite de almacenamiento para el prototipo académico
OpenAI API	Límite de tokens por minuto (TPM) en la capa gratuita o de evaluación	Errores 429 bajo carga de evaluación de múltiples proyectos simultáneos	Retry con backoff exponencial; alternativa con Gemini API gratuita como fallback en el adaptador IIAService
LangGraph	API en evolución activa con cambios incompatibles entre versiones minor	Incompatibilidad de código entre actualizaciones de la dependencia	Versión fijada en requirements.txt; pruebas de regresión antes de cada actualización del framework
Celery + Redis	Redis standalone puede perder tareas ante fallos de	Evaluaciones pendientes no	Configurar Redis con RDB + AOF persistence en el PMV3; monitoreo

Herramienta	Limitación identificada	Impacto potencial	Acción de mitigación adoptada
	proceso sin persistencia configurada	reencoladas tras un reinicio del servicio	de las tareas con Flower
Google Colab	Desconexión automática tras 12 h de inactividad; sin persistencia garantizada de la sesión	Pérdida de los resultados de la experimentación de embeddings	Guardar los artefactos en Google Drive; versionar los notebooks en GitHub al finalizar cada sesión de trabajo
SonarCloud	No detecta errores lógicos de negocio ni fallos de los prompts de los agentes de lenguaje	Cobertura parcial de la calidad del sistema en su conjunto	Complementar con Pytest para la lógica de negocio y con LangSmith para la calidad de los prompts y las respuestas
Docker Desktop	Alto consumo de RAM en equipos con menos de 8 GB en Windows con WSL2	Degradación del entorno de desarrollo de algunos integrantes del equipo	Activar el modo WSL2 con límite de memoria configurado; como alternativa, usar GitHub Codespaces para los integrantes con equipos limitados

5.3 Registro de decisiones de adaptación

PMV	Problema detectado	Herramienta / configuración original	Decisión de adaptación y resultado esperado
PMV 1	Precisión del RAG inferior al 94 % con corpus reducido	Búsqueda solo vectorial densa (similitud coseno)	Incorporar búsqueda híbrida BM25 + densa en Qdrant con re-ranking; meta: alcanzar el umbral de precisión en el PMV2 con el corpus completo
PMV 1	Experimentación de embeddings lenta sin GPU en el entorno local	Entorno local sin GPU dedicada	Migrar la experimentación a Google Colab con GPU T4 gratuita; reducción estimada del tiempo de evaluación de modelos de 4 h a 30 min
PMV 2	Bloqueo de la API REST durante la evaluación de agentes de forma síncrona	Llamadas síncronas a los agentes desde el hilo de FastAPI	Implementar Celery + Redis para procesamiento 100 % asíncrono; meta: tiempo de bloqueo de la interfaz de usuario de 0 s, verificado con Postman y Locust
PMV 3	Endpoints desprotegidos	Rutas de evaluación sin capa	Implementar OAuth2 + JWT con python-jose y roles de

PMV	Problema detectado	Herramienta configuración original /	Decisión de adaptación y resultado esperado
	detectados en la revisión de seguridad del equipo	de autenticación explícita en el PMV2	usuario; meta: 0 vulnerabilidades de autenticación en el escaneo con OWASP ZAP

6. Conclusiones

El ecosistema de herramientas seleccionado responde directamente a las necesidades técnicas del proyecto y cumple con los cuatro verbos exigidos por la competencia AG-I11 de ICACIT:

Crear: Se configuraron y construyeron soluciones específicas a partir de las herramientas seleccionadas: la arquitectura Docker Compose multi-servicio, el esquema de migraciones Alembic, la colección de pruebas Postman y los notebooks de experimentación de embeddings en Colab.

Seleccionar: Se evaluaron múltiples alternativas para cada necesidad mediante matrices ponderadas alineadas a las restricciones reales del proyecto: coste cero, compatibilidad con LangGraph y escalabilidad para el prototipo académico.

Aplicar: Las herramientas se aplicaron de forma integrada y progresiva en los tres PMV, demostrando un ecosistema de ingeniería cohesionado y no un uso aislado de herramientas individuales.

Reconocer limitaciones: Se documentaron formalmente las limitaciones de nueve herramientas críticas con sus impactos cuantificados y las acciones de mitigación implementadas o planificadas por PMV.

La selección tecnológica queda diferenciada en dos niveles. Mediante matrices ponderadas se seleccionaron FastAPI (4.80), PostgreSQL (4.75), Qdrant (4.80), LangGraph como orquestador principal (4.60) y OpenAI API como servicio LLM principal (4.35). CrewAI y Gemini API no se descartan: CrewAI se adopta como complemento para la definición y coordinación de agentes, mientras Gemini API permanece como alternativa/fallback del proveedor LLM. Como soporte del proyecto se adoptan Python y Uvicorn; SQLAlchemy y Alembic; Celery y Redis; Docker y Docker Compose; Git y GitHub; PlantUML/Draw.io, C4 y BPMN; Trello/GitHub Projects/Discord; y las herramientas de pruebas, observabilidad y seguridad descritas en la sección 3. Esta combinación responde a las necesidades de PMV1, PMV2 y PMV3, al enfoque de arquitectura hexagonal, al RAG Agéntico y a los requisitos de trazabilidad del HITL. En consecuencia, la selección no se presenta como arbitraria: las herramientas críticas fueron comparadas con criterios ponderados y las herramientas de soporte fueron adoptadas por su función técnica explícita dentro de la arquitectura.

Referencias bibliográficas

[1] FASTAPI. FastAPI Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://fastapi.tiangolo.com/>

[2] SQLALCHEMY. SQLAlchemy Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://docs.sqlalchemy.org/>

[3] LANGCHAIN. LangGraph documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://docs.langchain.com/oss/python/langgraph/overview>

[4] QDRANT. Qdrant Documentation: Hybrid Search and Hybrid Queries [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://qdrant.tech/documentation/search/hybrid-queries/>

[5] CELERY PROJECT. Celery Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://docs.celeryq.dev/>

[6] ICACIT. Criterios de acreditación y atributos del graduado [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://www.webicacit.com/>

[7] DOCKER, INC. Docker Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://docs.docker.com/>

[8] SONARSOURCE. SonarQube Cloud Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://docs.sonarsource.com/sonarqube-cloud/>

[9] OWASP FOUNDATION. OWASP Top 10 [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://owasp.org/www-project-top-ten/>

[10] OPENAI. OpenAI API Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://platform.openai.com/docs/>

[11] CREWAI. CrewAI Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://docs.crewai.com/>

[12] OPENAI. Structured Outputs and function calling. OpenAI API Reference [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://platform.openai.com/docs/api-reference/>

[13] GOOGLE. Gemini API: Function calling [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://ai.google.dev/gemini-api/docs/function-calling>

[14] POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://www.postgresql.org/docs/>

[15] ALEMBIC. Alembic Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://alembic.sqlalchemy.org/>

[16] REDIS. Redis Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://redis.io/docs/latest/>

[17] GIT. Git Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://git-scm.com/doc>

[18] GITHUB. GitHub Docs [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://docs.github.com/>

[19] PYTEST. pytest Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://docs.pytest.org/>

[20] POSTMAN. Postman Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://learning.postman.com/docs/>

[21] LOCUST. Locust Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://docs.locust.io/>

[22] OWASP FOUNDATION. ZAP Documentation [en línea]. [Consulta: 18 septiembre 2026]. Disponible en: <https://www.zaproxy.org/docs/>